

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA : Bambang Darmawan Saputra

NIM : 109082530003

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pengurutan data (sorting) merupakan salah satu operasi fundamental dalam ilmu komputer dan pemrograman yang bertujuan untuk menyusun kumpulan acak elemen data ke dalam urutan tertentu. Pengurutan dapat dilakukan secara membesar (ascending), yaitu menyusun data dari nilai terkecil menuju nilai terbesar, maupun secara mengecil (descending), dari nilai terbesar menuju terkecil. Proses ini sangat krusial karena sekumpulan data yang telah terurut akan mempermudah dan mempercepat berbagai operasi lanjutan, seperti pencarian data (searching), analisis algoritma, serta penyajian informasi agar lebih terstruktur dan mudah dipahami.

Salah satu algoritma pengurutan dasar yang umum dipelajari adalah Selection Sort. Algoritma ini bekerja dengan konsep membagi array atau rentang data menjadi dua bagian imajiner: bagian yang sudah terurut dan bagian yang belum terurut. Pada setiap iterasinya, algoritma akan memindai sisa data yang belum terurut untuk mencari nilai ekstrim (nilai minimum untuk urutan ascending atau maksimum untuk descending). Setelah nilai ekstrim tersebut ditemukan, elemen itu akan ditukar tempatnya (swap) dengan elemen terdepan dari bagian yang belum terurut. Proses pencarian dan pertukaran ini terus diulang secara sekuensial hingga seluruh elemen berpindah ke posisi urutan yang benar.

Selain Selection Sort, terdapat juga algoritma Insertion Sort yang memiliki pendekatan mekanis berbeda. Insertion Sort bekerja dengan logika yang menyerupai cara seseorang mengurutkan sekumpulan kartu remi di tangannya. Algoritma ini memproses data satu per satu dengan cara mengambil sebuah elemen dari bagian yang belum terurut, kemudian membandingkannya dengan elemen-elemen sebelumnya. Jika elemen tersebut lebih kecil (dalam kasus ascending), maka elemen-elemen sebelumnya yang lebih besar akan digeser ke kanan untuk membuka ruang, dan elemen yang sedang diproses disisipkan ke posisi yang tepat. Algoritma ini dikenal sangat efisien dan adaptif apabila kumpulan data masukan sudah dalam kondisi hampir terurut.

B. Guided

1. [guided1]

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string
```

```

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrString
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 12 - Modu
l 14\guided\guided.go"
5
jeruk
apel
mangga
pisang
anggur
anggur apel jeruk mangga pisang

```

Penjelasan :

Program ini membaca sejumlah kata sesuai input n, lalu mengurutkan kata-kata tersebut secara leksikografis menggunakan algoritma insertion sort yang bekerja langsung pada array data setiap kata berikutnya disimpan sementara di temp, dibandingkan dengan kata-kata sebelumnya, dan dipindahkan ke posisi yang benar sehingga hasil akhirnya adalah daftar kata terurut.

2. [guided2]

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan bulat
       F.S. array T terurut secara mengecil atau descending dengan
       INSERTION SORT */

    var temp, i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n int

    fmt.Scan(&n)
```

```

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSort(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 12 - Modul 14\guided\tempCodeRunnerFile.go"
5
12
7
23
4
15
23 15 12 7 4

```

Penjelasan :

Program ini membaca n bilangan bulat, lalu mengurutkannya secara menurun menggunakan insertion sort yang memindahkan setiap elemen ke posisi yang benar dengan membandingkan ke elemen sebelumnya; setelah fungsi insertionSort selesai, main mencetak deretan bilangan dalam urutan descending.

3. [guided3]

```

package main

import "fmt"

func selectionSortArray(angka *[5]int){
    var idx_min, i, j int
    for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi sortingnya)
        idx_min = i
        for j = i + 1; j < len(angka); j++ { //loop dalam (mencari nilai ekstrim)
            if angka[j] <= angka[idx_min] { //sorting terkecil ke terbesar

```

```

        idx_min = j
    }
}
if idx_min != i {
    angka[i], angka[idx_min] = angka[idx_min], angka[i]
}
}
}

func main(){
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
    selecetionSortArray(&arrAngka)
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 12 - Modu
l 14\guided\tempCodeRunnerFile.go"
Masukkan data angka ke-0 : 29
Masukkan data angka ke-1 : 10
Masukkan data angka ke-2 : 17
Masukkan data angka ke-3 : 5
Masukkan data angka ke-4 : 23

=== SEBELUM DISORTING ===
29 - 10 - 17 - 5 - 23 -
=== SETELAH DISORITNG ===
5 - 10 - 17 - 23 - 29 -

```

Penjelasan :

Program ini membaca 5 bilangan dari pengguna ke dalam array arrAngka, lalu mengurutkannya naik dengan selection sort dengan mencari elemen

terkecil di setiap iterasi dan menukarnya ke posisi yang benar, kemudian menampilkan array yang sudah terurut.

4. [guided4]

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrString
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}
```

```
}
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 12 - Modul 14\guided\tempCodeRunnerFile.go"
4
apel
pisang
anggur
jeruk
anggur apel jeruk pisang
```

Penjelasan :

Program ini membaca n string dari input, lalu mengurutkannya secara leksikografis naik menggunakan insertion sort yang menyimpan elemen saat ini di temp, menggeser elemen sebelumnya yang lebih besar ke kanan sampai posisi yang tepat ditemukan, dan akhirnya mencetak string yang sudah terurut.

C. Unguided

1. Soal Unguided 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Jawaban:

```
package main

import "fmt"

// Fungsi algoritma selection sort membesar (ascending)
func selectionSortAsc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
```



```

        idxMin = j
    }
}
// Proses pertukaran nilai (swap)
arr[i], arr[idxMin] = arr[idxMin], arr[i]
}
}

func main() {
    var n int
    fmt.Scan(&n) // Membaca jumlah daerah

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m) // Membaca jumlah rumah di daerah tersebut
        arr := make([]int, m)

        for j := 0; j < m; j++ {
            fmt.Scan(&arr[j])
        }

        selectionSortAsc(arr, m)

        for j := 0; j < m; j++ {
            fmt.Print(arr[j], " ")
        }
        fmt.Println()
    }
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 12 - Modu
1 14\unguided\tempCodeRunnerFile.go"
2
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189

```

Penjelasan :

Program ini membaca sejumlah daerah dan jumlah rumah di masing-masing daerah, kemudian mengurutkan nomor rumah kerabat secara membesar (ascending) menggunakan algoritma Selection Sort. Algoritma ini berulang kali mencari nilai terkecil dari sisa data yang belum terurut , kemudian

menukarnya dengan elemen paling depan dari bagian rentang yang belum terurut tersebut.

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Jawaban :

```
package main

import "fmt"

func selectionSortAsc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        arr[i], arr[idxMin] = arr[idxMin], arr[i]
    }
}

func selectionSortDesc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMax := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[idxMax] {
                idxMax = j
            }
        }
        arr[i], arr[idxMax] = arr[idxMax], arr[i]
    }
}

func main() {
    var n int
```

```

fmt.Scan(&n)

for i := 0; i < n; i++ {
    var m int
    fmt.Scan(&m)
    var ganjil, genap []int

    for j := 0; j < m; j++ {
        var val int
        fmt.Scan(&val)
        if val%2 != 0 {
            ganjil = append(ganjil, val)
        } else {
            genap = append(genap, val)
        }
    }

    selectionSortAsc(ganjil, len(ganjil))
    selectionSortDesc(genap, len(genap))

    // Cetak ganjil terlebih dahulu, kemudian genap
    for _, val := range ganjil {
        fmt.Print(val, " ")
    }
    for _, val := range genap {
        fmt.Print(val, " ")
    }
    fmt.Println()
}
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 12 - Mo
1 14\unguided\unguided2.go"
2
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189

```

Penjelasan :

Program ini memisahkan nomor rumah bernilai ganjil dan genap ke dalam dua penampungan (array) yang berbeda saat data dibaca. Setelah dipisah, bilangan ganjil diurutkan secara membesar (ascending) dan bilangan genap

diurutkan secara mengecil (descending) menggunakan algoritma Selection Sort. Program kemudian mencetak deretan nomor tersebut dengan menampilkan kelompok bilangan ganjil terlebih dahulu, dilanjutkan dengan kelompok bilangan genap

3. Soal Unguided 3

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Jawaban :

```
package main

import "fmt"

// Fungsi algoritma insertion sort membesar
func insertionSort(arr []int, n int) {
    for i := 1; i < n; i++ {
        temp := arr[i]
        j := i
        for j > 0 && temp < arr[j-1] {
            arr[j] = arr[j-1]
            j--
        }
        arr[j] = temp
    }
}

func main() {
    var arr []int
    var val int

    // Membaca masukan hingga ditemukan bilangan negatif
    for {
        fmt.Scan(&val)
        if val < 0 {
            break
        }
        arr = append(arr, val)
    }

    insertionSort(arr, len(arr))

    // Menampilkan hasil
    for _, v := range arr {
        fmt.Print(v, " ")
    }
    fmt.Println()

    // Menampilkan status jarak
    for i := 1; i < len(arr); i++ {
        if arr[i] - arr[i-1] == arr[i-1] - arr[i-2] {
            fmt.Print("Data berjarak", arr[i-1] - arr[i-2], " ")
        } else {
            fmt.Print("data berjarak tidak tetap ")
        }
    }
    fmt.Println()
}
```

```

    }
    arr = append(arr, val)
}

n := len(arr)
insertionSort(arr, n)

// Mencetak array yang telah terurut
for i := 0; i < n; i++ {
    fmt.Print(arr[i], " ")
}
fmt.Println()

// Memeriksa status jarak antar data
if n < 2 {
    fmt.Println("Data berjarak tidak tetap")
} else {
    diff := arr[1] - arr[0]
    isConstant := true
    for i := 2; i < n; i++ {
        if arr[i]-arr[i-1] != diff {
            isConstant = false
            break
        }
    }
    if isConstant {
        fmt.Printf("Data berjarak %d\n", diff)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 12 - Modu
l 14\unguided\unguided3.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap

```

Penjelasan :

Program ini membaca sekumpulan bilangan bulat non-negatif hingga menemukan bilangan negatif sebagai penanda untuk berhenti. Kumpulan data tersebut kemudian disisipkan ke posisi yang seharusnya agar terurut membesar menggunakan metode Insertion Sort. Terakhir, program

mengevaluasi apakah selisih atau jarak antar setiap bilangan yang saling berdekatan di dalam array selalu sama (konstan) atau berjarak tidak tetap.

4. Soal Unguided 4

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Jawaban :

```
package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating    int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)
    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
        &pustaka[i].penulis, &pustaka[i].penerbit,
        &pustaka[i].eksemplar, &pustaka[i].tahun,
        &pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }
    maxIdx := 0
```

```

    // Pencarian sekuensial untuk rating tertinggi pada data yang
    belum terurut
    for i := 1; i < n; i++ {
        if pustaka[i].rating > pustaka[maxIdx].rating {
            maxIdx = i
        }
    }
    fav := pustaka[maxIdx]
    fmt.Printf("%s, %s, %s, %d\n", fav.judul, fav.penulis,
fav.penerbit, fav.tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    // Insertion sort mengecil (descending) berdasarkan rating
    for i := 1; i < n; i++ {
        temp := pustaka[i]
        j := i
        for j > 0 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j--
        }
        pustaka[j] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5
    if n < 5 {
        limit = n
    }
    for i := 0; i < limit; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    left := 0
    right := n - 1
    found := false
    var mid int

    // Pencarian Biner (Binary Search)
    for left <= right && !found {
        mid = (left + right) / 2
        if pustaka[mid].rating == r {

```

```

        found = true
    } else if pustaka[mid].rating < r {
        // Karena array terurut mengecil, jika nilai tengah lebih
        kecil dari yang dicari,
        // maka cari di sisi kiri
        right = mid - 1
    } else {
        left = mid + 1
    }
}

if found {
    b := pustaka[mid]
    fmt.Printf("%s, %s, %s, %d, %d, %d\n", b.judul, b.penulis,
b.penerbit, b.tahun, b.eksemplar, b.rating)
} else {
    fmt.Println("Tidak ada buku dengan rating seperti itu")
}
}

func main() {
    var pustaka DaftarBuku
    var n, r int

    DaftarkanBuku(&pustaka, &n)
    CetakTerfavorit(pustaka, n)
    UrutBuku(&pustaka, n)
    Cetak5Terbaru(pustaka, n)

    // Membaca rating yang ingin dicari di akhir
    fmt.Scan(&r)
    CariBuku(pustaka, n, r)
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 12 - Modu
l 14\unguided\unguided4.go"
3
B01 Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 10 2005 5
B02 Bumi_Manusia Pramoedya_Ananta_Toer Hasta_Mitra 5 1980 4
B03 Cantik Itu Luka Eka_Kurniawan Gramedia 8 2002 3
Laskar_Pelangi, Andrea_Hirata, Bentang_Pustaka, 2005
Laskar_Pelangi
Bumi_Manusia
Cantik Itu Luka
4
Bumi_Manusia, Pramoedya_Ananta_Toer, Hasta_Mitra, 1980, 5, 4

```

Penjelasan :

Program ini bertugas mengelola data buku dengan membaca jumlah N buku beserta detail atributnya (id, judul, penulis, penerbit, eksemplar, tahun, rating), lalu mencetak data buku terfavorit pada data yang belum terurut. Setelah itu, program mengurutkan buku berdasarkan nilai rating secara menurun/mengecil menggunakan algoritma Insertion Sort, menampilkan 5 judul buku dengan rating tertinggi, dan melakukan pencarian biner (binary search) untuk menemukan detail lengkap sebuah buku berdasarkan angka rating spesifik yang diinput oleh pengguna pada baris terakhir.

D. Kesimpulan

Berdasarkan implementasi pada berbagai program di atas, dapat disimpulkan bahwa algoritma Selection Sort dan Insertion Sort merupakan metode yang sangat andal untuk mengelola dan mengurutkan struktur data array pada bahasa Go, baik untuk tipe data dasar maupun tipe data bentukan yang kompleks seperti struct. Keduanya memiliki karakteristik unik; Selection Sort unggul dalam logika pencarian nilai ekstrim yang konsisten, sementara Insertion Sort sangat efektif dalam menyisipkan elemen melalui pergeseran data. Pemahaman terhadap kedua algoritma ini memberikan fondasi logika yang kuat bagi programmer dalam menyusun data secara sistematis, yang pada akhirnya memungkinkan penerapan algoritma tingkat lanjut dengan optimal, seperti pencarian biner (binary search).